EXPERIMENT-3

NAME : SOAMESHWARAN D ROLL NO : 240701520

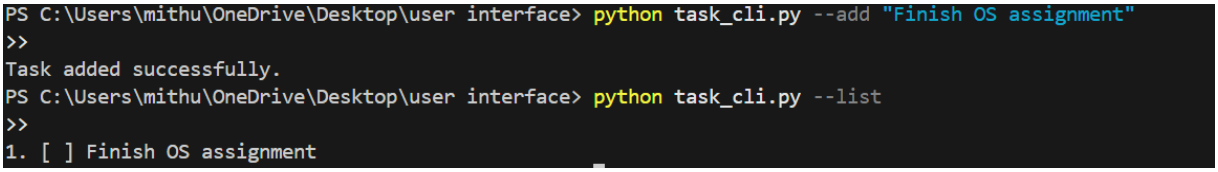
1.COMMAND LINE INTERFACE:

PYTHON CODE:

```
import argparse
import os
FILE_NAME = "tasks.txt"
def add_task(task):
    with open(FILE_NAME, "a") as f:
        f.write(f"[ ] {task}\n")
    print("Task added successfully.")
def list_tasks():
    if not os.path.exists(FILE_NAME):
        print("No tasks found.")
        return
    with open(FILE_NAME, "r") as f:
        tasks = f.readlines()
    for i, task in enumerate(tasks, start=1):
        print(f"{i}. {task.strip()}")
def complete_task(task_number):
    if not os.path.exists(FILE_NAME):
        print("No tasks to complete.")
        return
    with open(FILE_NAME, "r") as f:
        tasks = f.readlines()
    if task_number < 1 or task_number > len(tasks):
        print("Invalid task number.")
        return
    tasks[task_number - 1] = tasks[task_number - 1].replace("[ ]", "[✓]", 1)
    with open(FILE_NAME, "w") as f:
        f.writelines(tasks)
    print("Task marked as completed.")
def main():
```

```
    parser = argparse.ArgumentParser(description="Smart Task Tracker CLI")
    parser.add_argument("--add", type=str, help="Add a new task")
    parser.add_argument("--list", action="store_true", help="List all tasks")
    parser.add_argument("--done", type=int, help="Mark task as completed")
    args = parser.parse_args()
    if args.add:
        add_task(args.add)
    elif args.list:
        list_tasks()
    elif args.done:
        complete_task(args.done)
    else:
        parser.print_help()
if __name__ == "__main__":
    main()
```

OUTPUT:



2.GRAPHICAL USER INTERFACE:

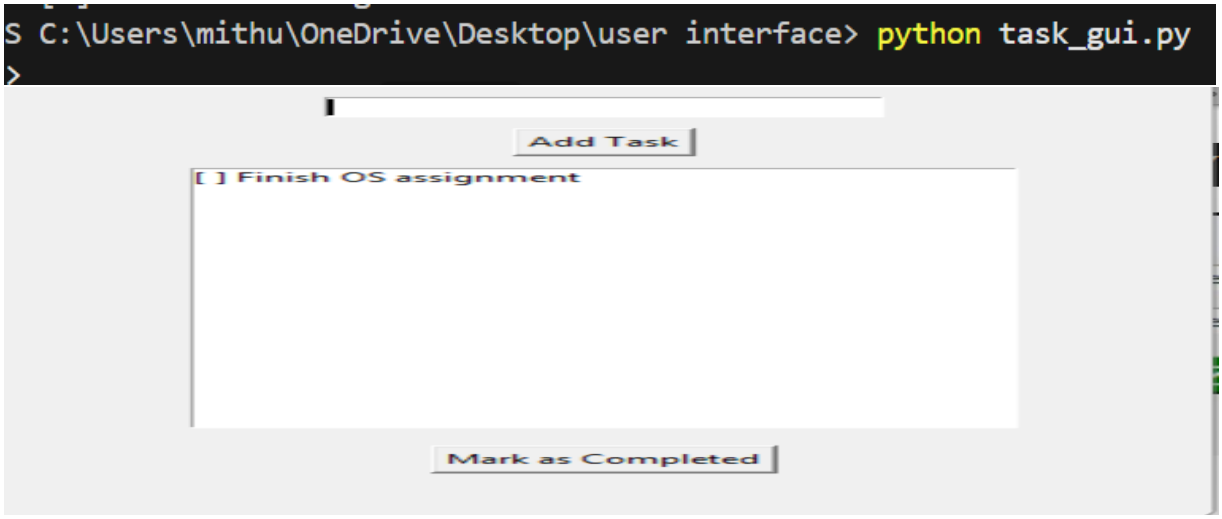
PYTHON CODE:

```
import tkinter as tk
from tkinter import
messagebox
import os
FILE_NAME = "tasks.txt"
def load_tasks():
    if
os.path.exists(FILE_NAM
E):
    with open(FILE_NAME,
"r") as f:        for task in f:
task_listbox.insert(tk.EN
D, task.strip())
```

```
def add_task():
    task = task_entry.get()
    if task == "":
messagebox.showwarning("Warning", "Task cannot be empty")
        return
task_listbox.insert(tk.END, "[ ] " + task)
    task_entry.delete(0, tk.END)
    save_tasks()
def complete_task():
    try:
        index = task_listbox.curselection()[0]
        task = task_listbox.get(index)
        if task.startswith("[✓]"):
            return
task_listbox.delete(index)
task_listbox.insert(index, task.replace("[ ]", "[✓]", 1))
        save_tasks()
    except:
messagebox.showwarning("Warning", "Select a task first")
def save_tasks():
    with open(FILE_NAME, "w") as f:
        for i in range(task_listbox.size()):
```

```
 f.write(task_listbox.get(i)
+ "\n")

root = tk.Tk()

root.title("Task Manager
GUI")

root.geometry("400x400
")

task_entry =
tk.Entry(root, width=30)

task_entry.pack(pady=10
)

add_button =
tk.Button(root,
text="Add Task",
command=add_task)

add_button.pack()

task_listbox =
tk.Listbox(root,
width=45, height=15)

task_listbox.pack(pady=1
0)

complete_button =
tk.Button(root,
text="Mark as
Completed",
command=complete_tas
k)

complete_button.pack(p
ady=5

load_tasks()

root.mainloop()root.mai
nloop()
```

OUTPUT:



3.VOICE USER INTERFACE:

PYTHON CODE:

```
import speech_recognition as sr
import pyttsx3
# Initialize text-to-speech engine
engine = pyttsx3.init()
engine.setProperty("rate", 170)
def speak(text):
    engine.say(text)
    engine.runAndWait()
def listen():
    recognizer = sr.Recognizer()
    with sr.Microphone() as source:
        print("Listening...")
        recognizer.adjust_for_ambient_noise(source)
        audio = recognizer.listen(source)
    try:
        command = recognizer.recognize_google(audio)
        print("You said:", command)
        return command.lower()
    except sr.UnknownValueError:
```

```
        return ""
    except sr.RequestError:
        speak("Speech service is unavailable")
        return ""
def main():
    speak("Voice assistant started")
    while True:
        command = listen()
        if "hello" in command:
            speak("Hello. How can I help you")
        elif "your name" in command:
            speak("I am your voice assistant")
        elif "exit" in command or "stop" in command:
            speak("Goodbye")
            break
        elif command != "":
            speak("Command not recognized")
if __name__ == "__main__":
    main()
```

output

```
PS C:\Users\mithu\OneDrive\Desktop\user interface> python voice_ui.py
>>
```

```
You said: sum of rupees
Listening...
You said: design
Listening...
Listening...
You said: hi
```

4.USER SATISFACTION COMPARISON:

PYTHON CODE:

```
import matplotlib.pyplot as plt
```

```
cli_scores = []
gui_scores = []
voice_scores = []
n = int(input("Enter number of users: "))
for i in range(n):
    print(f"\nUser {i+1} Ratings (1 to 5)")
    cli = int(input("CLI Satisfaction: "))
    gui = int(input("GUI Satisfaction: "))
    voice = int(input("Voice UI Satisfaction: "))
    cli_scores.append(cli)
    gui_scores.append(gui)
    voice_scores.append(voice)
cli_avg = sum(cli_scores) / n
gui_avg = sum(gui_scores) / n
voice_avg = sum(voice_scores) / n
print("\nAverage Satisfaction Scores")
print("CLI:", cli_avg)
print("GUI:", gui_avg)
print("Voice UI:", voice_avg)
interfaces = ["CLI", "GUI", "Voice UI"]
scores = [cli_avg, gui_avg, voice_avg]
plt.bar(interfaces, scores)
plt.xlabel("User Interface Type")
plt.ylabel("Average Satisfaction Score")
plt.title("User Satisfaction Comparison")
plt.ylim(0, 5)
plt.show()
```

output

```
PS C:\Users\mithu\OneDrive\Desktop\user interface> python user_satisfaction.py
>>
Enter number of users: 4

User 1 Ratings (1 to 5)
CLI Satisfaction: 5
GUI Satisfaction: 5
Voice UI Satisfaction: 5
```